

Teach Module 9: Build and Dependency Management with Maven

Module 9 focuses on **Build and Dependency Management with Maven**.

This guide teaches the topic from general Java and software engineering knowledge. The course document was used only to identify the module topic.

What Maven Is

Maven is a build tool for Java projects. Its job is to answer these questions:

- How do I compile this project?
- What libraries does it need?
- How do I run tests?
- How do I package it into a `.jar` or `.war` ?
- How can another machine build it the same way?

Think of Maven as the project manager for your Java application.

At the center of every Maven project is a file called:

```
pom.xml
```

`pom` means **Project Object Model**. It describes your project.

A very small Maven `pom.xml` looks like this:

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>hello-maven</artifactId>
  <version>1.0.0</version>
</project>
```

The three key identifiers are:

- `groupId` : Usually your organization or package-style name, like `com.company` .
- `artifactId` : The project, application, or library name, like `inventory-service` .
- `version` : The version of the project, like `1.0.0` .

Together, these identify the built artifact:

```
com.example:hello-maven:1.0.0
```

Standard Maven Project Structure

Maven expects a standard folder structure:

```
my-app/  
  pom.xml  
  src/  
    main/  
      java/  
        com/example/App.java  
      resources/  
    test/  
      java/  
        com/example/AppTest.java  
      resources/
```

Important folders:

- `src/main/java` : Production Java code.
- `src/main/resources` : Config files, properties files, templates, and other runtime resources.
- `src/test/java` : Test code.
- `src/test/resources` : Files needed only for tests.

This standard structure matters because Maven tools, IDEs, and CI systems all understand it automatically.

Maven Build Lifecycle

Maven builds projects using phases. The most common commands are:

```
mvn compile
```

Compiles your Java code.

```
mvn test
```

Compiles and runs tests.

```
mvn package
```

Builds the final artifact, usually a `.jar` or `.war`.

```
mvn clean
```

Deletes generated build files from the `target/` folder.

```
mvn clean package
```

Cleans the project and builds it fresh.

The important build folder is:

```
target/
```

That is where Maven puts compiled classes, test results, and packaged output.

Dependencies

Most real Java apps use external libraries. For example, JUnit is commonly used for testing.

Instead of manually downloading `.jar` files, you declare dependencies in `pom.xml` :

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Maven downloads the dependency for you from a repository, usually Maven Central.

The `scope` tells Maven when the dependency is needed.

Common scopes:

- `compile` : Needed for main application code. This is the default.
- `test` : Needed only for tests.
- `provided` : Needed to compile, but provided by the runtime environment.
- `runtime` : Not needed to compile, but needed when the app runs.

JAR vs WAR

A `.jar` file is usually used for standalone Java apps or Spring Boot apps.

```
inventory-service.jar
```

A `.war` file is used for traditional web apps deployed to an external servlet container like Tomcat.

```
inventory-webapp.war
```

In modern Java backend development, especially with Spring Boot, `.jar` packaging is very common.

Build Profiles

Profiles let you change build behavior for different environments.

Common profile names include:

- `dev`
- `test`
- `prod`

You might use profiles to change:

- database settings
- logging settings
- plugin behavior
- packaging behavior
- environment-specific resources

Run a Maven profile like this:

```
mvn package -Pdev
```

CI Integration

In CI/CD, Maven is commonly used like this:

```
mvn clean test
```

or:

```
mvn clean package
```

A CI pipeline can use Maven to make sure:

- the project compiles
- tests pass
- dependencies resolve correctly
- the final package can be built

That is why Maven is important professionally: it makes builds repeatable.

Quick Mental Model

When you run:

```
mvn clean package
```

Maven basically says:

1. Delete old build output.
2. Read `pom.xml` .
3. Download required dependencies.
4. Compile main code.
5. Compile test code.
6. Run tests.
7. Package the application.
8. Put output in `target/` .

Practice Exercises

Exercise 1: Create a Basic Maven Project

Create a simple Java Maven project with this structure:

```
maven-practice/  
  pom.xml  
  src/main/java/com/example/App.java
```

Add a class that prints:

```
Hello from Maven
```

Practice running:

```
mvn compile  
mvn package
```

Then inspect the `target/` folder.

Exercise 2: Add a Dependency

Add a dependency such as Apache Commons Lang:

```
<dependency>  
  <groupId>org.apache.commons</groupId>  
  <artifactId>commons-lang3</artifactId>  
  <version>3.14.0</version>  
</dependency>
```

Use `StringUtils.isBlank()` in your Java code.

```
String input = " ";  
System.out.println(StringUtils.isBlank(input));
```

Goal: understand how Maven downloads and connects external libraries.

Exercise 3: Add JUnit Tests

Add JUnit 5 to your `pom.xml` with `test` scope.

Create:

```
src/test/java/com/example/AppTest.java
```

Write tests for a small method, such as:

```
public static int add(int a, int b) {  
  return a + b;  
}
```

Run:

```
mvn test
```

Goal: understand test dependencies and the Maven test phase.

Exercise 4: Understand Dependency Scope

Create examples using these scopes:

```
<scope>compile</scope>
<scope>test</scope>
<scope>provided</scope>
```

Then answer:

- Should JUnit be available in production code?
- Should a servlet API be packaged inside a WAR?
- What happens if a dependency is missing from the right scope?

Exercise 5: Package as a JAR

Configure your project to produce a `.jar`.

Run:

```
mvn clean package
```

Find the generated file:

```
target/your-project-name-1.0.0.jar
```

Goal: understand Maven artifact generation.

Exercise 6: Create an Executable JAR

Use the Maven JAR plugin or Shade plugin to make a runnable JAR.

Then run:

```
java -jar target/your-project-name-1.0.0.jar
```

Goal: understand the difference between a normal JAR and an executable JAR.

Exercise 7: Explore Maven Lifecycle Phases

Run these one by one:

```
mvn validate
mvn compile
mvn test
mvn package
mvn install
```

After each command, inspect what changed in `target/`.

Goal: learn what each lifecycle phase actually does.

Exercise 8: Break the Build on Purpose

Introduce a syntax error in your Java code.

Run:

```
mvn compile
```

Then fix it.

Next, make a test fail and run:

```
mvn test
```

Goal: understand how Maven reports build and test failures.

Exercise 9: Use Maven Profiles

Create two profiles:

```
dev
prod
```

Use them to set different property values.

Run:

```
mvn package -Pdev
mvn package -Pprod
```

Goal: understand environment-specific builds.

Exercise 10: Create a Simple Multi-Class App

Build a small calculator project:

```
Calculator.java
App.java
CalculatorTest.java
```

Methods:

```
add()
subtract()
multiply()
divide()
```

Add tests for all methods.

Run:

```
mvn clean test
```

Goal: combine structure, testing, and Maven build commands.

Exercise 11: Analyze the Dependency Tree

Run:

```
mvn dependency:tree
```

Look at direct and transitive dependencies.

Goal: understand that one dependency can bring other dependencies with it.

Exercise 12: Simulate a CI Build

Pretend you are a CI server and run:

```
mvn clean verify
```

Goal: practice the command commonly used before merging or deploying code.

Lab: Student Grade Calculator with Maven

Goal

Build a small Java project using Maven, add dependencies, write tests, and package the app.

What You Will Practice

- Maven project structure
- `pom.xml`
- Dependencies
- JUnit testing
- Maven lifecycle commands
- Packaging a `.jar`
- Inspecting the `target/` folder

Step 1: Create the Project

Create this folder structure:

```
student-grade-calculator/  
  pom.xml  
  src/  
    main/  
    java/  
      com/example/grades/
```



```
App.java
GradeCalculator.java
test/
  java/
    com/example/grades/
      GradeCalculatorTest.java
```

Step 2: Create pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>student-grade-calculator</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Step 3: Create GradeCalculator.java

```
package com.example.grades;

public class GradeCalculator {

    public double average(int[] scores) {
        if (scores == null || scores.length == 0) {
            throw new IllegalArgumentException("Scores cannot be empty");
        }

        int total = 0;

        for (int score : scores) {
```

```

        total += score;
    }

    return (double) total / scores.length;
}

public int highest(int[] scores) {
    if (scores == null || scores.length == 0) {
        throw new IllegalArgumentException("Scores cannot be empty");
    }

    int highest = scores[0];

    for (int score : scores) {
        if (score > highest) {
            highest = score;
        }
    }

    return highest;
}

public boolean isPassing(double average) {
    return average >= 70.0;
}
}

```

Step 4: Create App.java

```

package com.example.grades;

public class App {

    public static void main(String[] args) {
        GradeCalculator calculator = new GradeCalculator();

        int[] scores = {85, 90, 78, 92};

        double average = calculator.average(scores);
        int highest = calculator.highest(scores);
        boolean passing = calculator.isPassing(average);

        System.out.println("Average: " + average);
        System.out.println("Highest: " + highest);
        System.out.println("Passing: " + passing);
    }
}

```

Step 5: Create GradeCalculatorTest.java

```
package com.example.grades;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class GradeCalculatorTest {

    @Test
    void averageReturnsCorrectValue() {
        GradeCalculator calculator = new GradeCalculator();

        int[] scores = {80, 90, 100};

        assertEquals(90.0, calculator.average(scores));
    }

    @Test
    void highestReturnsLargestScore() {
        GradeCalculator calculator = new GradeCalculator();

        int[] scores = {72, 88, 95, 81};

        assertEquals(95, calculator.highest(scores));
    }

    @Test
    void isPassingReturnsTrueForAverageAbove70() {
        GradeCalculator calculator = new GradeCalculator();

        assertTrue(calculator.isPassing(75.0));
    }

    @Test
    void averageThrowsExceptionForEmptyArray() {
        GradeCalculator calculator = new GradeCalculator();

        assertThrows(IllegalArgumentException.class, () -> {
            calculator.average(new int[] {});
        });
    }
}
```

Step 6: Run Maven Commands

From inside the project folder, run:

```
mvn compile
```

Then:

```
mvn test
```

Then:

```
mvn clean package
```

Step 7: Inspect Output

Look inside:

```
target/
```

You should see a generated file similar to:

```
student-grade-calculator-1.0.0.jar
```

Step 8: Practice Questions

Answer these after completing the lab:

1. What file does Maven read to understand your project?
2. What folder contains production Java code?
3. What folder contains test code?
4. Why does JUnit use `<scope>test</scope>` ?
5. What does `mvn clean package` do?
6. Where does Maven place compiled and packaged output?

Challenge Task

Add a new method:

```
public int lowest(int[] scores)
```

Then add a unit test for it and run:

```
mvn test
```

Quick Check Answers

- Maven reads `pom.xml` .
- Production Java code goes in `src/main/java` .
- Test Java code goes in `src/test/java` .
- JUnit uses `test` scope because it is needed only for testing, not production runtime.
- `mvn clean package` deletes old build output and creates a fresh packaged artifact.
- Maven places compiled and packaged output in `target/` .